Kente Retail — Order Service Handover

Onboarding & Branching Recommendation | Prepared by Moses Furaha | moses.furaha@amalitech.com | September 3, 2026

Part 1 — Onboarding Note for the Next Hire

Context: the previous ops-adjacent engineer left without notice or handover notes. A release was blocked because nobody could confirm the state of the repository or the sandbox server. This note exists so the next person never inherits that situation blind.

The DevOps Lifecycle, in Plain Language

DevOps is a continuous loop, not a one-time handoff: **Plan** → **Code** → **Build** → **Test** → **Release** → **Deploy** → **Operate** → **Monitor**, then back to Plan. Every stage feeds the next, and monitoring feeds back into planning. The point of the loop is that no single person has to hold the whole system in their head — the process and the tooling carry that knowledge instead. This incident happened precisely because the loop broke down at the handoff points: nothing was documented between "Operate" and the next person's "Plan."

CALMS: the Five Pillars

Culture	Shared ownership over one person's tribal knowledge. A single engineer being the only one who understood the repo and server is a culture failure, not just bad luck.
Automation	Repeatable scripts/CI checks (branch protection, secret scanning, health checks) reduce reliance on any one person remembering a manual step.
Lean	Small, frequent changes are easier to hand over and easier to undo than large, long-lived branches like the tangled history found here.
Measurement	You can only tell something is wrong (or fixed) if you're tracking metrics — see DORA metrics below.
Sharing	Documentation, runbooks, and this onboarding note itself are how knowledge survives someone's sudden departure.

How This Incident Shows Up in DORA Metrics

Change Failure Rate — the repo had an unresolved merge conflict and a secret committed into history. Had that been deployed as-is, it would very likely have caused a failed or rolled-back release, directly increasing change failure rate. **Mean Time to Restore (MTTR)** — because there were no handover notes, the time to restore the release to a working, deployable state was inflated by the time spent simply figuring out what state the system was in, before any actual fix could start. **Deployment Frequency** took a direct hit too: Monday's release was blocked outright, not just slowed down.

Takeaway for the next hire: treat a missing or incomplete handover as a DORA-metric risk, not just an inconvenience — it degrades the same numbers leadership already tracks.

Part 2 — Branching Strategy Recommendation

Recommendation: Git Flow

Adopt Git Flow (main / develop / feature/* / hotfix/*) for this team going forward, rather than trunk-based development. This is a direct response to what actually went wrong here, not a textbook default.

- **The tangled branch history and unresolved merge conflict** happened because there was no agreed structure for what a branch was for or when it should merge. Git Flow fixes that by giving every branch a defined role and a defined lifetime, so "why does this branch still exist" stops being a mystery six months later.
- **The leaked secret** reached history through an ordinary commit on a long-lived branch with no gate in front of it. Git Flow's feature-branch-plus-PR pattern gives us a natural place to enforce branch protection and pre-merge secret scanning before anything touches *develop* or *main*.
- **The existing** `hotfix/timeout-bug` **branch** created during this handover is itself evidence Git Flow already fits how this team responds to live incidents — an isolated hotfix branch cut from main, fixed, and merged back without disrupting in-progress feature work.

Trunk-based development was considered but set aside for now: it assumes strong CI gating and small, frequent merges as the default team habit, and this team just demonstrated the opposite habit (large, unreviewed, long-lived branches). Git Flow's explicit branch roles are the more realistic fit until that habit changes; trunk-based can be revisited once branch protection, PR review, and CI checks are consistently in place.

Reference: Kente Retail server-baseline-policy.md and Lab Brief “The Handover” — DevOps Foundations & Linux Essentials.